Health Risk Advice Implementation Report

Scope

This report explains the advice generation part for the diabetes, stroke, and hypertension risk prediction feature.

The advice module does not predict disease risk by itself. The prediction models first produce a risk result. The advice layer then explains what the user can do next using trusted health guidance and the latest prediction result.

1. Advice Architecture

The advice feature uses a controlled RAG architecture.

RAG means Retrieval-Augmented Generation. In this project, the backend first retrieves trusted health guidance chunks, then the ML service generates a short answer using only those chunks and the user's prediction context.

The architecture is:

```
Frontend or assistant chat
  -> Node/Express backend advice route
  -> latest prediction lookup
  -> trusted guidance retrieval
  -> FastAPI health-advice endpoint
  -> controlled LLM answer
  -> safety note and sources returned
  -> chat history saved
```

2. Main Files

Frontend and chat entry:

```
frontend/src/screens/AssistantChatScreen.js
frontend/src/services/diabetesPredictionService.js
frontend/src/services/strokePredictionService.js
frontend/src/services/hypertensionPredictionService.js
```

Backend:

```
backend/src/routes/healthAdvice.js
backend/src/routes/predict.js
backend/src/services/healthAdviceService.js
backend/src/services/assistantOrchestrator.js
backend/src/data/healthAdviceKnowledge.js
backend/migrations/1747300000000_health_advice_rag.js
```

ML service:

```
ml-service/app/routers/assistant.py
ml-service/app/services/health_advice_answer.py
ml-service/app/services/ollama_client.py
```

3. Entry Points

Advice can be triggered from:

```
POST /api/health-advice
POST /predict/diabetes/chat
POST /predict/stroke/chat
POST /predict/hypertension/chat
main assistant chat
```

The dedicated advice route calls buildHealthAdviceResponse. The prediction chat routes also call the same advice service when the message is asking for advice instead of asking for a new prediction.

4. Intent Detection

The backend checks if the message is a health advice question.

Risk words include:

```
diabetes, diabetic, sugar, glucose, hypertension,
blood pressure, bp, pressure, stroke, risk
```

Advice words include:

```
advice, reduce, lower, prevent, what should, how can,
caregiver, monitor, guidance, manage, control, healthy, lifestyle
```

Example advice questions:

```
How can I reduce my diabetes risk?
What should I do about high blood pressure?
Why is my stroke risk high?
What should my caregiver monitor?
```

Medication-only questions are not handled by this advice module unless they also include diabetes, stroke, hypertension, or risk context.

5. Risk Type Selection

After detecting advice intent, the backend selects one risk type:

```
Diabetes
Stroke
Hypertension
```

If the question names a condition, that condition is selected. If the question is general, the backend checks the latest saved predictions and chooses the highest-risk condition.

The internal risk priority is:

```
High = 3
Medium = 2
Low = 1
Unknown = 0
```

This lets a broad question like "What should I do now?" use the most important available risk result.

6. Prediction Context

The advice service loads or receives the latest prediction context.

The context can include:

```
riskLevel
confidence
selectedAlgorithm
factors
summary
```

This allows the answer to refer to the user's actual risk level and the main contributing factors, such as high BMI, high blood pressure, high glucose, smoking history, or low physical activity.

7. Trusted Knowledge Base

Trusted advice chunks are defined in:

```
backend/src/data/healthAdviceKnowledge.js
```

The knowledge base contains guidance from:

```
WHO
CDC
American Heart Association
Sri Lanka NCD Unit
```

Chunks are grouped as:

```
General
Diabetes
Stroke
Hypertension
```

The database migration 1747300000000_health_advice_rag.js creates the health_advice_chunks table and inserts the same trusted chunks with embeddings.

If the table is unavailable, the backend falls back to the in-code chunks.

8. Retrieval Method

The backend builds a retrieval query from:

```
risk type
user question
latest risk level
prediction factors
```

Each chunk has a lightweight local embedding. The embedding method tokenizes text, removes

stop words, hashes tokens into a 64-dimensional vector, and normalizes the vector.

The retrieval score is:

```
final score = 0.7 * cosine similarity
            + 0.2 * term overlap
            + 0.1 * exact risk type boost
```

The default maximum retrieved chunks is:

```
6
```

This can be configured with:

```
HEALTH_ADVICE_MAX_CHUNKS
```

9. LLM Answer Generation

When chunks are found, the backend calls:

```
POST /assistant/health-advice
```

The payload contains:

```
question
risk_type
prediction
retrieved_chunks
user_role
```

The Python ML service then calls Ollama through ollama_client.py.

The health advice prompt is strict. It instructs the model to:

```
Use only retrieved trusted guidance and the prediction result.
Do not diagnose diseases.
Do not prescribe medicine.
Do not suggest stopping, changing, or starting medicine.
Do not replace professional healthcare advice.
Recommend professional consultation for high risk or abnormal readings.
Write 3 to 6 short sentences.
Return exactly one JSON object.
```

This means the LLM is used for wording and explanation, not for independent medical decision-making.

10. Fallback Logic

If the ML service fails, Ollama is unavailable, or the answer is empty, the backend creates a fallback answer.

The fallback uses:

```
risk type
risk level
top prediction factors
```

```
retrieved guidance topics
professional follow-up sentence
```

If no useful guidance chunks are available, the system returns:

```
Reliable guidance is not available in the current knowledge base for this question.
```

This makes the advice feature reliable even when the LLM dependency is down.

11. Response Format

The backend returns:

```
answer
riskType
prediction
sources
safetyNote
conversationId
fallback
followUps
```

The sources field lists the trusted sources used. The fallback field shows whether the backend had to use the deterministic fallback answer.

The safety note is:

```
This is general health guidance, not a medical diagnosis.
It does not replace advice from a healthcare professional.
```

12. Conversation Storage

Advice interactions are saved under the conversation title:

```
Health Risk Advice
```

The backend saves:

```
user question
assistant answer
retrieved source rows
intent = health_risk_advice
latency
fallback reason when applicable
```

This supports traceability because the app can later show what was asked, what advice was returned, and which chunks were used.

13. Why This Architecture Was Selected

The advice feature uses RAG instead of a direct free-form chatbot because health advice needs stronger control.

This design was selected because:

- It reuses the latest prediction result instead of guessing risk.
- It grounds advice in trusted health guidance.
- It returns source links.
- It restricts the LLM from diagnosis and medication instructions.
- It supports both elder and caregiver wording.
- It still returns a fallback answer when the LLM fails.
- It keeps the same advice pipeline reusable for diabetes, stroke, and hypertension.

14. Final Summary

The advice module is a controlled RAG layer built on top of the diabetes, stroke, and hypertension prediction modules.

The prediction models answer "What is the risk?". The advice module answers "What should the user know or do next?".

Technically, the backend detects advice intent, selects a risk type, loads the latest prediction, retrieves trusted guidance chunks, calls the FastAPI health-advice endpoint, and returns a short safe answer with sources and a medical safety note.

This architecture improves usability while keeping medical safety boundaries clear.